

Práctica 2: Análisis de algoritmos de ordenamiento.

Abigail Sánchez

Julio 2025

1 Introduction

Para la práctica, se analizan tres algoritmos de ordenamiento: bubble, selection e insertion sort, con el objetivo de estudiar su tiempo de ejecución desde un enfoque tanto teórico como experimental. Específicamente para bubble e insertion sort, el tiempo $T(n, k)$ como función del tamaño del arreglo n , y la cantidad de elementos inicialmente ordenados k , considerando que la primera mitad está ordenada y la segunda desordenada. Así se puede identificar cómo influye el grado de orden previo en el rendimiento, y si este puede llegar a utilizarse para optimizar el desempeño de los algoritmos.

Y para selection sort, concretamente el mejor y peor caso, para comparar si existe alguna diferencia importante entre ambos escenarios.

2 Marco Teórico

Los algoritmos de ordenamiento tienen como propósito poner en orden un conjunto de datos según algún criterio, normalmente de menor a mayor. En esta práctica nos enfocamos en tres de los algoritmos más conocidos y sencillos: Bubble Sort, Insertion Sort y Selection Sort. Todos ellos son llamados “directos” porque no usan estructuras auxiliares, sino que van organizando los elementos directamente sobre el mismo arreglo que reciben.

Bubble Sort funciona revisando pares de elementos que están uno junto al otro. Si detecta que están fuera de orden, los intercambia. Esta revisión se repite una y otra vez hasta que el arreglo queda completamente ordenado.

Insertion Sort, por otro lado, va tomando los elementos uno por uno y los va colocando donde corresponde, dentro de la parte del arreglo que ya está ordenada. Ambos algoritmos tienen algo en común: cuando el arreglo está muy desordenado, pueden tardar bastante en ordenarlo, con una complejidad de $O(n^2)$. Pero si el arreglo ya está en orden, o casi en orden, pueden terminar mucho más rápido, alcanzando incluso una complejidad de $O(n)$.

Selection Sort, en cambio, se comporta distinto. Lo que hace es buscar el elemento más pequeño del arreglo en cada iteración y colocarlo en su posición final. Repite este proceso hasta que todo el arreglo está ordenado. Este algoritmo siempre hace la misma cantidad de comparaciones, sin importar si los datos ya están ordenados o no. Por eso, su tiempo de ejecución no mejora en los mejores casos, y su complejidad sigue siendo $O(n^2)$ tanto en el mejor como en el peor escenario.

Para entender mejor cómo reaccionan estos algoritmos ante arreglos con diferentes niveles de orden, introducimos una nueva variable: k . Esta representa la cantidad de elementos que ya están ordenados desde el inicio del arreglo. Así, el tiempo de ejecución se puede expresar como $T(n, k)$, donde n es el tamaño total del arreglo. Esta forma de análisis nos permite ir más allá de los clásicos “mejor” y “peor” caso, y explorar cómo se comportan los algoritmos cuando el arreglo no está completamente ordenado, pero tampoco completamente revuelto.

3 Resultados

Para analizar experimentalmente el comportamiento de los algoritmos de ordenamiento *Bubble Sort* e *Insertion Sort*, se generaron gráficas de $T(n, k)$ manteniendo el tamaño del arreglo n fijo y variando el número de elementos inicialmente ordenados k .

3.1 Bubble sort

Podemos ver que su rendimiento mejora conforme aumenta la cantidad de elementos que ya están ordenados. En otras palabras, mientras más ordenado esté el arreglo desde el inicio, menos comparaciones necesita hacer el algoritmo. Si dejamos el tamaño del arreglo (n) fijo y solo variamos cuánto está ordenado (k), el tiempo que tarda el algoritmo disminuye de forma constante, casi como una línea recta que baja. Por eso, una **regresión lineal** es una buena forma de representar este comportamiento en las gráficas de Bubble Sort cuando n es constante.

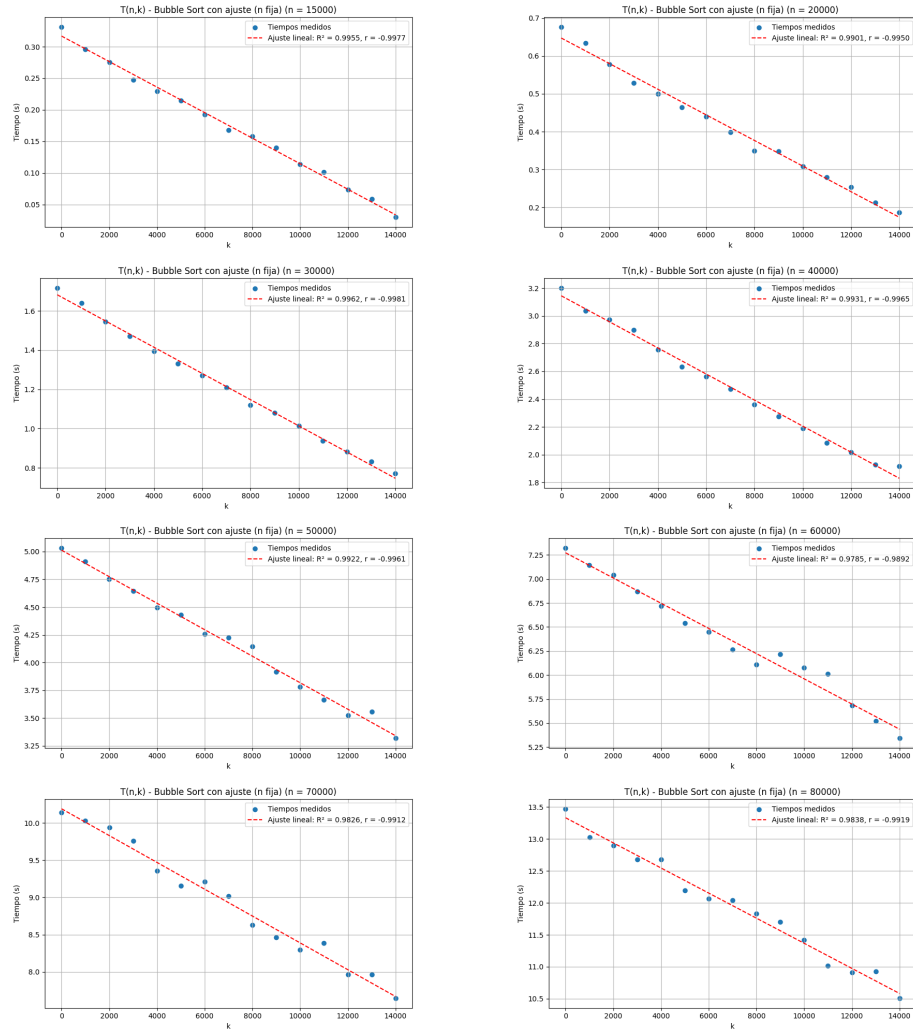


Figure 1: Resultados de Bubble Sort ($n = 15,000$ a $80,000$)

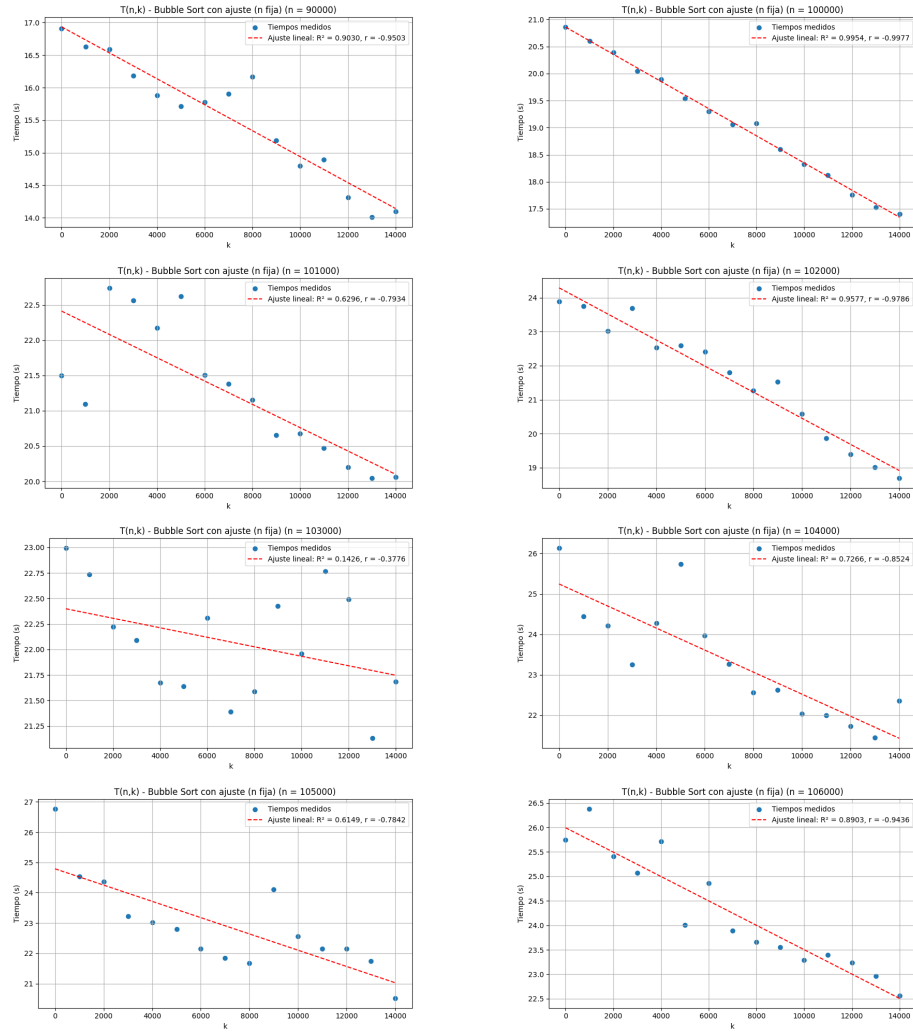


Figure 2: Resultados de Bubble Sort ($n = 90,000$ a $106,000$)

3.2 Insertion Sort

En este caso, el algoritmo es aún más sensible al orden previo del arreglo. Si los primeros k elementos ya están ordenados pero el resto no, el algoritmo necesita hacer más trabajo con cada elemento desordenado, insertándolos poco a poco en su lugar. Esto provoca que el tiempo de ejecución crezca de forma más compleja, no lineal, sino más parecida a una curva.

Cuando mantenemos n fijo y observamos cómo cambia el tiempo según el valor de k , notamos que la relación sigue una **forma de parábola decreciente**. Es decir, a medida que hay más elementos ordenados, el tiempo baja, pero lo hace siguiendo una curva. Por eso, en este caso, una **regresión polinomial de segundo grado** es la mejor opción para ajustar los datos en las gráficas de Insertion Sort.

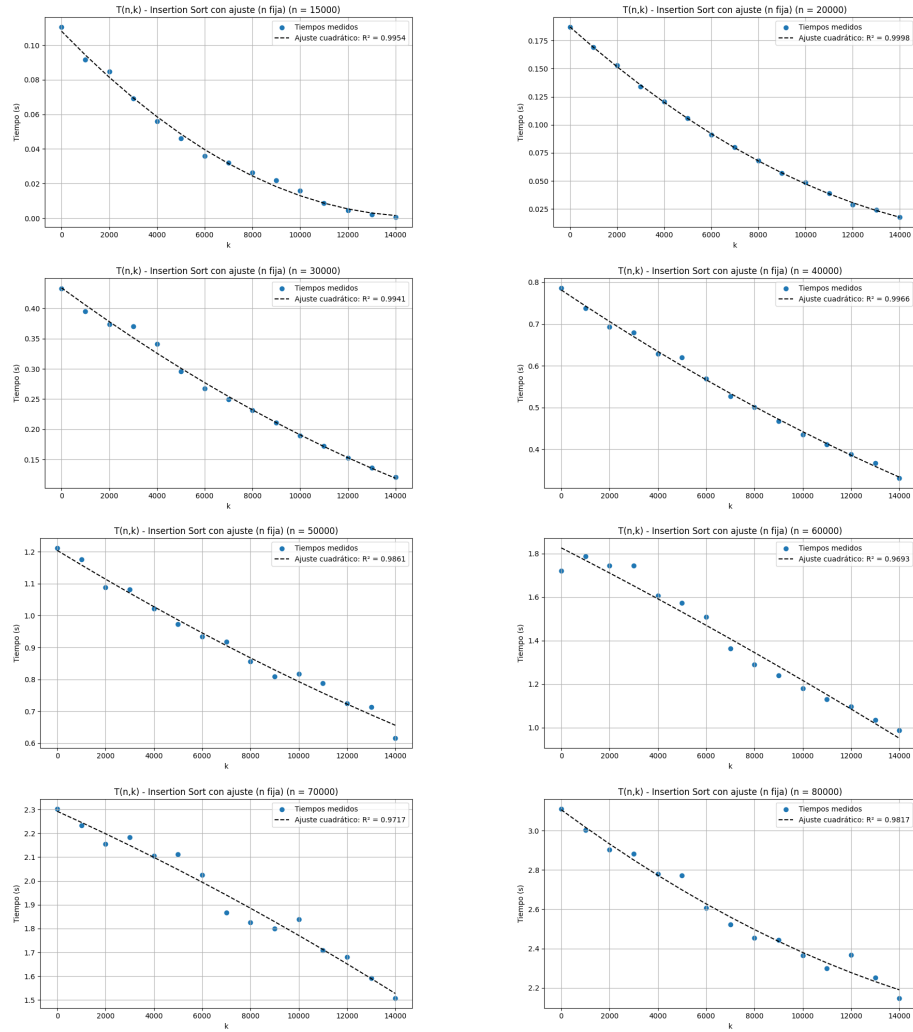


Figure 3: Resultados de Insertion Sort ($n = 15,000$ a $80,000$)

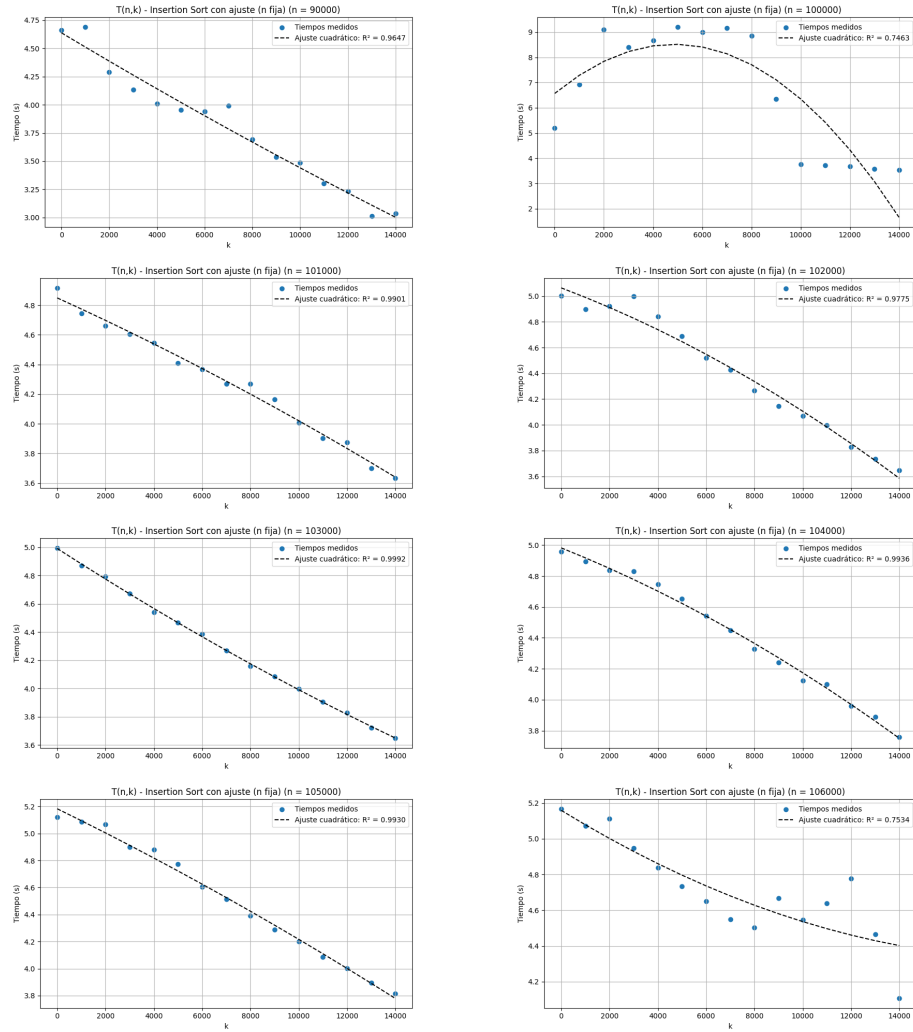


Figure 4: Resultados de Insertion Sort ($n = 90,000$ a $106,000$)

3.3 Selection sort

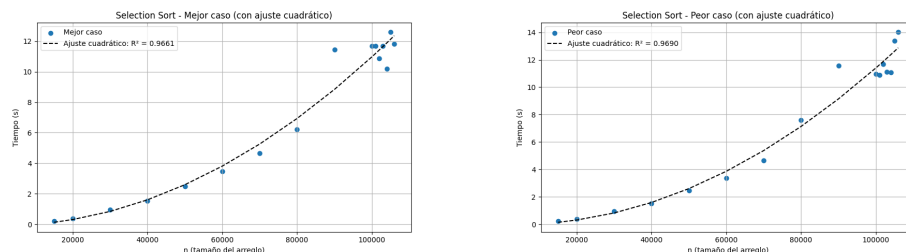


Figure 5: Resultados de selection sort.

En ambas gráficas, el tiempo de ejecución crece de forma parabólica con respecto al tamaño del arreglo, entonces esto confirma la complejidad teórica de $O(n^2)$ del algoritmo para ambos casos. Y las curvas son prácticamente iguales, validando que el orden inicial no afecta el rendimiento del algoritmo. Esto se debe a que siempre siempre recorre el arreglo completo para encontrar el mínimo.

4 Conclusión

La variable k , que representa cuántos elementos del arreglo ya están ordenados al principio, resultó muy útil para observar cómo se comportan los algoritmos cuando no partimos de un caso completamente aleatorio o completamente ordenado. Al variar k , podemos ver de forma más clara cómo reacciona cada algoritmo ante distintos grados de orden, y además, si tomamos $k = 0$, recuperamos automáticamente el peor caso posible.

En el caso de *Bubble Sort*, notamos que los primeros k elementos —por estar ya en orden— nunca se intercambian. Eso hace que, en la práctica, el algoritmo actúe solo sobre los $n - k$ elementos restantes. Aun así, como el algoritmo recorre todo el arreglo para no asumir nada, termina teniendo un término proporcional a $k \cdot n$ en su función de tiempo. Si dejamos fijo n , este término se vuelve lineal con respecto a k , pero en general sigue teniendo comportamiento cuadrático.

Con *Selection Sort*, el comportamiento es distinto. El algoritmo siempre recorre todo el arreglo en busca del mínimo, sin importar si ya está ordenado o no. Además, aunque un elemento esté en su lugar, igual se hace un intercambio consigo mismo. Eso hace que el mejor y el peor caso se comporten igual, al menos en cuanto al número de operaciones.

Por otro lado, *Insertion Sort* sí se beneficia mucho del orden previo. Cuando encuentra una parte del arreglo ya ordenada, simplemente deja de comparar

y sigue con el siguiente elemento. En este caso, el tiempo de ejecución depende de cuántos elementos están desordenados al final, y eso genera un término cuadrático en $n - k$, que al fijar n se convierte en una función cuadrática decreciente respecto a k .

En resumen, aunque los tres algoritmos comparten una complejidad cuadrática en el peor caso, se comportan de maneras muy distintas cuando el arreglo ya viene parcialmente ordenado. Esta diferencia puede aprovecharse si tenemos alguna idea del estado inicial de los datos que queremos ordenar.